# FASTSPEECH 2 TEXT-TO-SPEECH SYNTHESIS (ENGLISH AND TELUGU)

*Technical Report*

**Author:** Sathvik Konduri **Date:** 7 November 2026

**Repository:** https://github.com/aquasp3/fastspeech2-dual-tts-english-telugu

## 1. IMPLEMENTATION APPROACH & ARCHITECTURE

The implementation is based on the ming024/FastSpeech2 PyTorch repository: a feed-forward transformer encoder and decoder, a variance adaptor with duration, pitch (F0) and energy predictors, and a mel-spectrogram postnet. The architecture was kept at the reference defaults (4 encoder and 4 decoder layers, hidden size 256, about 29.3 million parameters). Two separate models were trained, one per language, each in its own folder with its own configuration.

### Modifications

Changes common to both languages: audio/stft.py was patched to select the device dynamically instead of the hard-coded .cuda(); preprocessor/preprocessor.py was patched so pitch/energy scalers and frame counts are updated only for utterances that have an alignment, and so the validation set is chosen by duration (val_hours); inference.py synthesises the five test sentences, saves wavs and plots predicted duration, pitch (de-normalised with stats.json) and energy per phone; checkpoints are stored as weights-only 12,000-step files. Telugu-specific: text/symbols.py (character-level symbols for U+0C00-U+0C7F, '@'-prefixed), text/telugu_text.py (normalisation, number/time/ordinal expansion, one phone per character, 'sp' at word boundaries) and gen_telugu_textgrid.py (proportional alignment with weights: vowel signs 1.2, virama 0.25, anusvara-type signs 0.7, space 0.6, others 1.0). English-specific: gen_english_textgrid.py (MMS_FA word alignment, intra-word phones weighted vowels 1.5, consonants 1.0).

## 2. DATASETS, SUBSAMPLING STRATEGY & PREPROCESSING

English (Svarah): A random ~6.2 h subset (seed 1234) was drawn from Svarah and force-aligned; 5.74 h remained after alignment. After alignment and preprocessing, 4,157 utterances remained: 3,657 for training (5.05 h) and 500 for validation (0.69 h). Training and validation do not overlap.

Telugu (IndicTTS Telugu, speaker1): From 8,576 available utterances, a random subset (seed 1234) of utterances between 1 and 11 seconds was selected, up to a 6-hour target. The 11-second cap keeps mel length under the maximum sequence length of 1000 frames. After alignment, 3,167 utterances (5.02 h) remained: 2,545 for training (4.02 h) and 622 for validation (1.00 h), chosen by duration after a seeded shuffle. Training and validation do not overlap.

Preprocessing (both): Audio at 22,050 Hz, 80 mel channels, hop length 256, window and FFT size 1024, mel range 0 to 8000 Hz. Pitch is extracted with PyWorld (DIO plus StoneMask) and energy is the L2 norm of each STFT frame. Both are averaged per phone and normalised with training statistics (Telugu pitch mean 190.4 Hz, standard deviation 46.5 Hz).

Training (both): Adam (betas 0.9 and 0.98), batch size 16, 12,000 steps, 2,000 warm-up steps, learning-rate decay by 0.3 at steps 7,200, 9,600 and 10,800, gradient clipping 1.0, validation every 500 steps, checkpoints every 2,000 steps, on an NVIDIA RTX 3050 (8 GB).

Deviations from the brief: the brief asks for about 5 h training and 1 h validation per language. English training is 5.05 h but validation is 0.69 h. Telugu validation is 1.00 h but training is 4.02 h, because only 5.02 h remained after the 1-11 s length filter and removal of failed alignments. The 12,000-step budget was chosen to fit the deadline; the brief gives no step count.

## 3. ANALYSIS OF TRAINING & VALIDATION LOSS CURVES

| Step | English Train Loss | English Val Loss | Telugu Train Loss | Telugu Val Loss |
|---|---|---|---|---|
| 100 | 8.29 | — | 9.87 | — |
| 500 | 4.14 | 4.32 | 4.34 | 4.13 |
| 2,000 | 3.87 | 4.35 | 2.89 | 3.80 |
| 4,000 | 3.86 | 4.00 | 2.04 | 3.90 |
| 6,000 | 3.14 | 3.97 | 1.83 | 4.04 |
| 10,000 | 2.31 | 4.19 | 1.24 | 4.06 |
| 12,000 | 1.87 | 4.22 | 1.33 | 4.07 |

Convergence: English training loss fell from 8.29 to about 4 within 500 steps and stayed near 3.9 to 4.4 until about step 4,000, then declined steadily to 1.87. Telugu training loss fell faster, reaching 2.04 at step 4,000 and a minimum of 1.09 at step 10,300. Both are dominated by the mel terms.

Instability: The training curves are jittery because each point is one batch: 18 of 120 English points and 4 of 120 Telugu points are more than 15% above the previous point. No divergence occurred, and no validation point jumped by more than 15%.

Overfitting: Both models overfit mildly. English validation loss reached its minimum of 3.97 at step 6,000 and rose to 4.22 while training loss kept falling. Telugu validation loss reached its minimum of 3.78 at step 1,500 and rose slowly to 4.07, so overfitting began much earlier, and its final gap between training and validation loss (1.33 against 4.07) is larger than English's (1.87 against 4.22). Both gaps are consistent with a small single-speaker training set of about 4 to 5 hours. An earlier checkpoint would have had lower validation loss; the 12,000-step checkpoint was kept because validation loss alone does not measure intelligibility.

Telugu duration loss: The final Telugu validation duration loss (0.024) is almost ten times lower than English (0.19). This does not mean accurate durations. Proportional alignment makes the targets a near-deterministic function of character type and utterance length, so they are easy to predict but are not real speech timings.

## 4. OBSERVATIONS ON GENERATED SPEECH

### 4.1 English Test Sentences

The English test sentences exercise specific front-end behaviours. E1 contains a very long word handled by the G2P fallback. E2a and E2b contain 'conduct' as a noun and as a verb; the front-end looks words up without part-of-speech information, so any stress difference depends on the pronunciation returned for the word. E3 contains the split token 'rhyth m', kept exactly as given, which the front-end treats as two words with a pause between them. E4 contains a time, a date and an out-of-vocabulary word ('andaroo'); numbers are expanded by the English cleaner and the unknown word goes through G2P. Audio for all five sentences is in Results/English/test_generated_audios.

### 4.2 Telugu Test Sentences

The Telugu output is only partially intelligible. Some characters and syllables are audible, but words are not consistently clear and the audio is not clean. All five sentences show this limitation. T1 (67 characters) is a long technical compound, where proportional alignment has the most room for error. T2a and T2b contain the same word with two meanings (grip and silk); the model has no way to distinguish them beyond the characters. T3 is the shortest sentence (32 characters). T4 (115 characters) is the longest; the year, date and time are expanded into Telugu number words before synthesis, so the digits are present in the input. The phone counts (T1 67, T2a 33, T2b 38, T3 32, T4 115) confirm that no characters were dropped. The Telugu sentences in the assignment PDF contained text-layer corruption (misplaced vowel signs and spacing) and were retyped to the intended spelling.

## 5. ANALYSIS OF VARIANCE ADAPTOR PLOTS

Plots for each test sentence are in Results/*/variance_adaptor_plots (duration, pitch, energy). Duration: Telugu duration targets come from fixed character weights, so predicted bars follow character class (very short for the virama, longer for vowel signs, a distinct bar at each word boundary), and longer sentences (T1, T4) show more bars. This regularity reflects the alignment heuristic, not learned prosody. English durations come from word-level forced alignment and therefore vary with real timing. Pitch: values are de-normalised to Hz (Telugu mean 190.4 Hz, standard deviation 46.5 Hz) and are averaged per phone, so contours are smooth and do not contain zeros inside voiced stretches. Energy: also averaged per phone, with low values expected at pause tokens. In Telugu these averages inherit the approximate phone boundaries, which tends to flatten the contours.

## 6. COMPARATIVE OBSERVATIONS

Both models use the same architecture, optimiser settings, HiFi-GAN vocoder and step budget, so differences come mainly from the text front-end and the alignments.

- English durations come from MMS_FA forced word alignment, with estimated phone boundaries inside each word. Telugu durations come from purely proportional character splitting. This is the main difference between the pipelines and the most likely reason for Telugu's reduced clarity.
- English uses phonemes with a lexicon; Telugu uses characters, a weaker representation of pronunciation.
- Both models overfit mildly, but Telugu's validation loss turned upward earlier (step 1,500 against step 6,000).

Failures: Telugu speech is only partially intelligible, and its very low duration loss reflects artificial targets. Both validation sets are smaller than or different from the brief (0.69 h for English, and 4.02 h of Telugu training audio).

In practice the Telugu speech is only partially intelligible, which is a larger quality gap than the loss values suggest, since Telugu training loss is lower than English. Both validation losses rose after early training (English after step 6,000, Telugu after step 1,500), indicating overfitting on the small training sets.

Possible improvements: apply the MMS_FA forced alignment used for English to Telugu characters; use a Telugu grapheme-to-phoneme lexicon; more data and more speakers; earlier checkpoint selection by validation loss; regularisation; and objective evaluation with MOS or an ASR-based character error rate.

## 7. REPRODUCIBILITY DETAILS

| Language Path | Configuration File | Checkpoint (Git LFS) | Inference Command |
|---|---|---|---|
| `English/` | `config.yaml` | `checkpoints/12000.pth.tar` | `python inference.py --restore_step 12000` |
| `Telugu/` | `config.yaml` | `checkpoints/12000.pth.tar` | `python inference.py --restore_step 12000` |

Software: Windows 11, RTX 3050 (8 GB), Python 3.11, PyTorch 2.5.1 (CUDA 12.1), setuptools 69.5.1, numpy 1.26.4; see requirements.txt.

Telugu (inside Telugu/): `python gen_telugu_textgrid.py --hours 6.0 ; python preprocess.py config/preprocess.yaml ; python train.py -p config/preprocess.yaml -m config/config.yaml -t config/train.yaml ; python plot_losses.py ; python inference.py --restore_step 12000`

English (inside English/): `python gen_english_textgrid.py` ; then the same preprocess, train, plot_losses and inference commands with the English config files.